CPE 314 — Software Design

University of Cebu

Computer Engineering Department

Trivia & Quiz Night Manager

Project Design Document

Group Name / No.:	2
Members:	John Daevin S. Simbajon - (Team Lead + Integrator/DevsOps)
	Clarence Philip A. Elegino - (Core OOP/Logic Developer)
	Justine B. Cuizon - (Database Admin + Data Developer + API)
	Joshua Neil G. Malaya - (UI/UX Designer + JavaFX Developer)
	Gebrielle P. Mirasol - (Analytics + QA)
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	*V0.2 - 10/28/26*

Table of Contents

Trivia & Quiz Night Manager *1*

Table of Contents *2*

Revision History *3*

Formatting Guidelines ..... ***Error! Bookmark not defined.***

1. Introduction & Project Overview *3*

2. Preliminary ER Sketch & Team Roles *4*

3. System Architecture & Object-Oriented Design *5*

4. Data Model & Database Design *5*

5. API Integration Design *5*

6. Concurrency Design *5*

7. Data Mining & Visualization Design *5*

8. Testing Summary *6*

9. Individual Contributions *6*

Appendix *6*

Revision History

Update this table each week as your team adds or revises sections of this document.

Version	Date	Authors	Changes
0.1	10/24/26-10/26/26	Justine B. Cuizon, Clarence A. Elegino, Joshua G. Malaya, Gebrielle P. Mirasol, John S. Simbajon	Initial draft — Week 8 sections (Section 1 to 2) Removing Unnecesary Instruction Generating the Table of Contents
0.2			Week 9 sections added
0.3			Week 10 sections added
1.0			Final compilation and polish — Week 11

1. Introduction & Project Overview

- Trivia & Quiz Game Night Manager is a Java desktop application that helps friends, classmates, and community groups organize and run trivia nights. Players enter their names, choose a category, and answer questions fetched live from the Open Trivia Database, with an “undo last answer” option for mistakes. Every player, game session, and score is saved in a local SQLite database, and past score history can be imported from CSV files. The app then analyzes the saved results and shows average score per category, most-missed questions, and score trends over time as labeled bar and line charts. Theme: Custom idea (an entertainment and social-gaming tool for trivia nights).

  Problem solved: Quiz hosts usually keep score on paper or in scattered spreadsheets, so results are easy to lose and hard to compare from one night to the next.

  Key value proposition: One app that fetches questions, scores each player, stores the history, and turns it into clear charts, so groups can see who is improving and which categories are hardest.

2. Preliminary ER Sketch & Team Roles

Figure 1: Entities, Key Attributes and Relationships

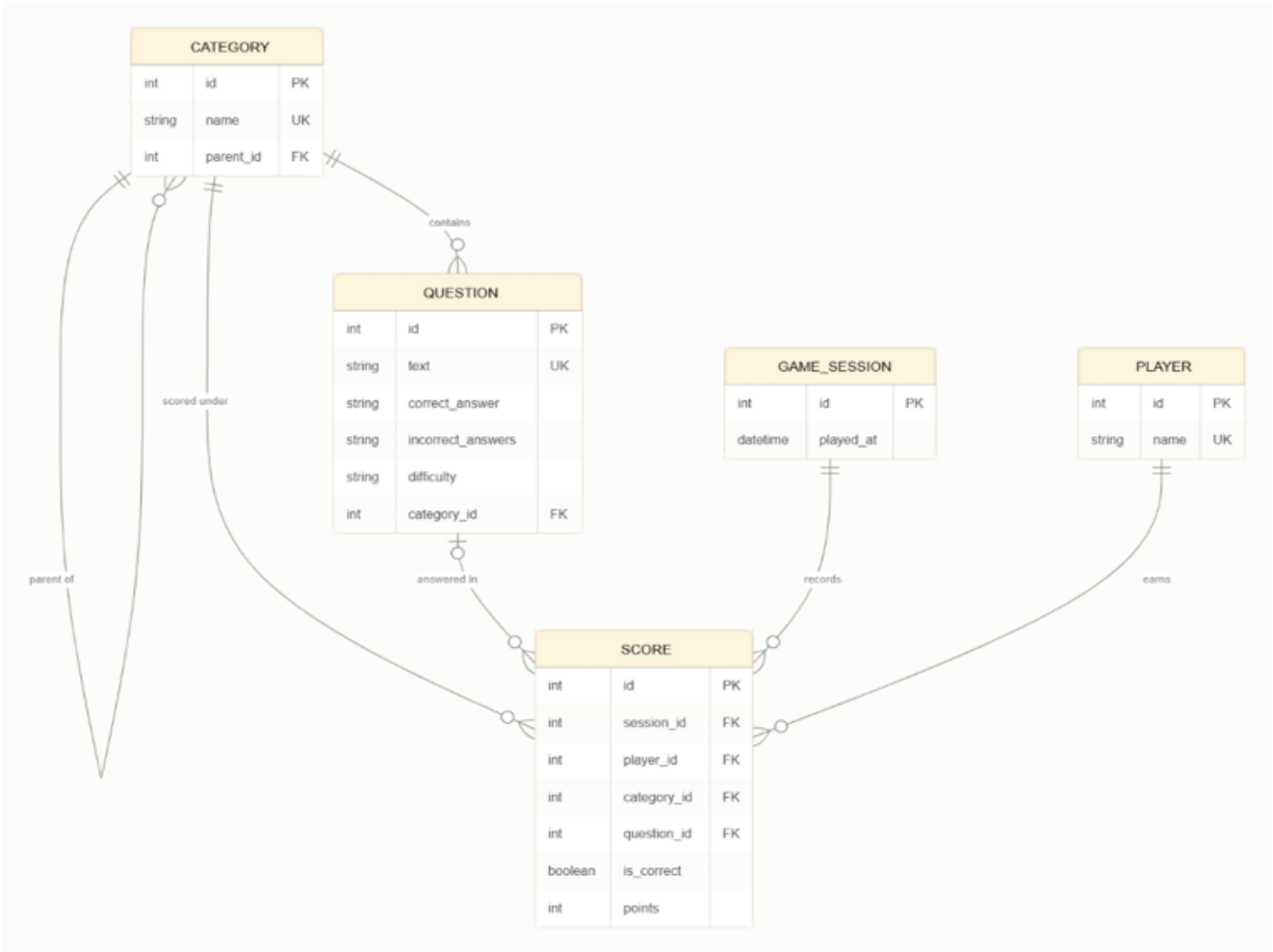


Table 1: Team members and primary roles

Member	**Primary Role**
John Daevin S. Simbajon	Team Lead + Integrator/DevOps
Gebrielle P. Mirasol	Core OOP/Logic Developer
Justine B. Cuizon	Database Admin + Data Developer + API
Clarence Philip A. Elegino	UI/UX Designer + JavaFX Developer
Joshua Malaya	Analytics + QA

3. System Architecture & Object-Oriented Design

Present your class hierarchy (a diagram or a clear textual description) showing the abstract class/interface and its concrete subclasses, and justify how each class follows the Single Responsibility Principle. Explain the design rationale in a short paragraph.

4. Data Model & Database Design

Include the finalized ER diagram (3+ entities, at least one relationship with correct cardinality) and the actual SQLite schema (CREATE TABLE statements or an equivalent schema table). This section supersedes the preliminary sketch from Section 2.

5. API Integration Design

Name the public API your app consumes, describe the endpoint(s) called, the shape of the JSON response, and how that data is parsed and used meaningfully in the app.

6. Concurrency Design

Identify the long-running operation(s) that run on a background thread, explain the threading approach used, and describe how the UI stays responsive during that operation.

7. Data Mining & Visualization Design

Describe the aggregation or frequency-analysis step performed on your app's data and the chart(s) used to visualize the result, including why that chart type was chosen.

8. Testing Summary

Summarize your JUnit test suite — how many tests exist, what they cover, and at least one meaningful edge case per tested class. A short table or list of key test cases is helpful here.

9. Individual Contributions

List what each team member specifically built or owned, referencing their git commit history, so the instructor can cross-check contribution during the Individual Q&A.

Appendix

(Add any additional references, links to full API documentation, licensing notes, or supplementary material here.)